NIELIT ROPAR

Web Scraping Simplified

A Comprehensive Guide to Extracting Data from the Web using Python & Flask

Access this slide online

Presented by Lovnish Verma | NIELIT Ropar/Chandigarh

Workshop Agenda

Part 1

Fundamentals & Ethics

Understanding Scraping vs. Crawling, Real-world use cases, the Legal Landscape, and HTTP/HTML Basics.

Part 2

The Python Ecosystem

Deep dive into the requests library and BeautifulSoup4. Parsing the DOM, finding elements, and structuring data.

Part 3

Flask Integration & Deployment

Building a Web UI for our scraper, handling user inputs, project architecture, and deploying via Docker/Hugging Face.

Part 1

Fundamentals & Ethics

What is Web Scraping?

Web scraping is the automated process of extracting unstructured data from websites and transforming it into structured formats.

- **Programmatic:** Done via code, not manual copy-pasting.
- **Targeted:** Extracts specific data points (e.g., prices, titles).
- **Scalable:** Can process thousands of pages in minutes.

Unstructured HTML → Structured Data

HTML → Python Dictionary → CSV / JSON / SQL Database

Crawling vs. Scraping

Web Crawling

Goal: Discovery & Indexing.

- Navigates across the web by following hyperlinks.
- Focuses on the "whole" page layout.
- E.g., Googlebot, Bingbot.

Web Scraping

Goal: Extraction & Formatting.

- Targets specific elements on specific pages.
- Focuses on extracting precise data points.
- E.g., Price monitoring scripts.

Why Scrape? (Use Cases)

Industry	Application	Value Generated
E-commerce	Price Monitoring & Competitor Analysis	Dynamic pricing strategies, identifying market gaps.
Finance	Aggregating News & Sentiment Analysis	Predicting stock movements based on public sentiment.
Real Estate	Property Listings Aggregation	Building central databases for buyers and renters.
Machine Learning	Creating Custom Datasets	Training LLMs and computer vision models on diverse data.
Marketing	Lead Generation	Collecting contact info from public directories.

Ethics & Legalities

- **Respect robots.txt:** The internet's standard for indicating what is allowed to be crawled. Check domain.com/robots.txt.
- **Identify Yourself:** Use transparent User-Agent strings. Include a contact email in the headers.
- **Rate Limiting:** Do not DDoS the server. Add sleep intervals (time.sleep(1)) between requests.
- **Public Data Only:** Avoid scraping behind logins unless authorized. Do not scrape PII (Personally Identifiable Information).

```
# Example of a robots.txt file
User-agent: *
Disallow: /admin/
Disallow: /private-api/
Allow: /public-data/
Crawl-delay: 10
```

Example:
https://www.nielit.gov.in/robots.txt

Understanding HTTP Responses

Before parsing HTML, we must successfully fetch it via an HTTP GET request.

200
OK
Request successful. Ready to parse.

403
Forbidden
Server refused. Often anti-bot protection.

404
Not Found
The URL does not exist or changed.

429
Too Many Requests
Rate limited. You are scraping too fast.

Pro Tip: Always use response.raise_for_status() in your code to catch errors immediately!

📡 **Understanding HTTP Response Status Codes — A Developer's Guide:**

https://lovnish.medium.com/understanding-http-response-status-codes-a-developers-guide-7bd4fb796f7d

Part 2

The Python Ecosystem

Core Python Libraries

Requests

The industry standard for making HTTP requests in Python. Handles cookies, sessions, and headers effortlessly.

`pip install requests`

BeautifulSoup4

A library for pulling data out of HTML and XML files. It creates a parse tree that makes navigating the DOM simple.

`pip install beautifulsoup4`

Selenium / Playwright

Browser automation tools. Required when scraping modern websites that rely heavily on JavaScript rendering.

`Advanced Use Case`

Step 1: Fetching the Data

```python
import requests

# 1. Define URL and Headers to mimic a real browser
url = "https://example.com"
headers = {"User-Agent": "Mozilla/5.0 ... "}

# 2. Make the GET request
response = requests.get(url, headers=headers)

# 3. Check for errors
response.raise_for_status()

# 4. Access the raw HTML
html_content = response.text
```

Why Headers Matter

Many servers block generic requests. By sending a **User-Agent** string, you tell the server:

"I am accessing this via Chrome on a Windows PC."

This significantly reduces the chance of receiving a **403 Forbidden** error.

Step 2: Parsing with BeautifulSoup

```
from bs4 import BeautifulSoup

# Pass the raw HTML to BeautifulSoup
soup = BeautifulSoup(html_content, "html.parser")

# 'html.parser' is Python's built-in parser

# The 'soup' object now represents the document
# as a nested data structure
```

The Parse Tree

BeautifulSoup transforms a complex HTML document into a tree of Python objects. The most common objects are:

Tag: Corresponds to an HTML tag.

T

NavigableString: The text content.

Step 3: Extracting Elements

```python
# Find a single element
title_tag = soup.find("h1")
print(title_tag.get_text()) # Extracts clean text

# Find all elements of a specific tag
paragraphs = soup.find_all("p")
elements = [e.get_text() for e in soup.find_all(tag)]

# Finding by Class or ID
prices = soup.find_all("span", class_="product-price")
product = soup.find("div", id="main-product")
```

The Core Methods

BeautifulSoup provides two primary methods for navigating the parse tree and locating specific data points.

find(): Returns the first matching element found in the document.

find_all(): Returns a list of all matching elements.

Flexible Queries: Search by tags, classes, or IDs.

App Logic: This powers our main scraping tool!

The two most powerful methods in BS4 are `find()` and `find_all()`.

Part 3

Flask Integration & Deployment

Why Flask?

From Script to Web App

A Python script runs in the terminal. Flask allows us to wrap our scraping logic into a web server, making it accessible via a browser, with a UI for inputs.

- **Micro-framework:** Lightweight and requires minimal boilerplate.
- **Routing:** Easily map URLs (e.g., /scrape) to Python functions.
- **Jinja2:** Render dynamic HTML templates based on Python variables.

Project Architecture

A standard Flask project requires a specific directory structure to locate static files and templates.

```
# Project Structure

web-scraper/
  ├── app.py # Main Flask logic
  ├── requirements.txt # Dependencies
  ├── Dockerfile # Container config
  └── templates/ # HTML Files
    ├── index.html # Input Form
    └── result.html # Output Display
```

Dependencies (requirements.txt)

flask
The core web framework and server

requests
Standard HTTP client for fetching web pages

bs4 (BeautifulSoup4)
Library for parsing and extracting data from HTML

gunicorn
Production-grade WSGI server for deployment

The Frontend (index.html)

We need an HTML form that sends a POST request containing the target URL and HTML Tag to our Flask backend.

```
<!-- index.html -->
<form action="/scrape" method="POST">
  <label for="url">Enter URL:</label>
  <input type="url" name="url" required>
  <label for="tag">Enter Tag to Scrape (e.g., p, h1):</label>
  <input type="text" name="tag" required>
  <button type="submit">Scrape</button>
</form>
```

The Backend Logic (app.py)

The Flask backend processes the URL and tag inputs, scrapes the target site, and returns parsed data to the results page.

```
@app.route("/scrape", methods=["POST"])
def scrape(): # 1. Retrieve user inputs
url = request.form.get("url")
tag = request.form.get("tag")
# 2. Validation check
if not url or not tag:
return render_template("result.html", error="Both required.")
# 3. Perform the scrape
response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(response.text, "html.parser")
# 4. Extract data
elements = [e.get_text() for e in soup.find_all(tag)]
# 5. Pass data back to frontend
return render_template("result.html", tag=tag, url=url, elements=elements)
```

Presenting Data (result.html)

Using Jinja2 templating syntax `{{ variable }}` and `{% control logic %}`, we dynamically render the list of scraped items.

```
<!-- result.html -->
<h3>Scraped Elements for Tag: {{ tag }}</h3>
<ul>
  {% for element in elements %}
    <li>{{ element }}</li>
  {% else %}
    <li>No content found for tag {{ tag }}.</li>
  {% endfor %}
</ul>
```

Containerization (Docker)

Docker packages our app and all its dependencies into a single container, ensuring it runs exactly the same way everywhere ("It works on my machine" solved).

```
FROM python:3.9 # Create user and set environment
USER user
WORKDIR /app

# Install dependencies
COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app code and start via Gunicorn
COPY --chown=user . /app
CMD ["gunicorn", "app:app", "-b", "0.0.0.0:7860"]
```

Gunicorn

Replaces Flask's built-in dev server with a production-ready Web Server Gateway Interface (WSGI).

Deploying to Production

Hugging Face Spaces

Ideal for machine learning apps, but supports standard Docker SDKs. Exposes port 7860 by default (which is why our Dockerfile maps to it).

hf.space

Render / Pythonanywhere

Platform-as-a-Service (PaaS) providers that allow seamless Github integration. Auto-builds and deploys upon code commits.

.onrender.com

Advanced Scraping Tactics

Challenge	Solution
IP Blocking / Rate Limiting	Use **Rotating Proxies** to change your IP address continuously.
CAPTCHAs	Integrate third-party captcha solving APIs (e.g., 2Captcha) or use undetectable browser modes.
Dynamic Content (React/Vue)	The HTML is empty until JS runs. Use **Selenium** or **Playwright** to load the JS before scraping.
Hidden APIs	Inspect Network Tab. The website might be fetching JSON from an API. Scrape the API directly instead of the HTML!

The Golden Checklist

Do

- Handle exceptions (Try/Except blocks).
- Check robots.txt.
- Cache responses during dev to save bandwidth.
- Use generic User-Agents.

Don't

- Hammer the server concurrently.
- Scrape PII without consent.
- Rely on highly specific, brittle CSS selectors.
- Ignore HTTP error codes.

Live Demo

Let's interact with the deployed application.

github.com/lovnishverma/webscaraping-simplified

lovnishverma-webscrapingexample.hf.space

Questions?

Thank you for attending the workshop.

Lovnish Verma

NIELIT Chandigarh

nielit.gov.in/chandigarh